Checkpoint 1: Pure C++ Implementation of Graphics Techniques

Project Title: Smartflow Interactive 2D Traffic Simulation

Repository: **https://github.com/BilliartMuthoni/smartflow-cpp-interactive-2D-traffic-simulation**

Group Members Details

Student No.	Name	Role
165899	Macharia Grace Wairimu	Concept and introduction
164099	Mwangi Billiart B. Muthoni	Mathematics and algorithms
146412	Njugi Elijah Muiru	Core C++ and integration
168377	Kihumba Kevin Renana	Scene design and interface
152710	Kisotu Samuel Lemayian	Documentation and testing

1. Project Description

SmartFlow is a 2D traffic simulation of a single road junction. Vehicles approach from four directions, stop at traffic signals, and go straight or turn left or right. The project uses a familiar traffic scene to show how computer graphics techniques are used to draw and move objects.

For this checkpoint, we implemented the graphics algorithms in pure C++. Each algorithm calculates exactly which pixels to colour, and our code saves the result as an image file. Every algorithm is applied to a simplified part of the junction: roads, lane markings, traffic lights, cars, buildings, trees, and vehicle routes.

2. Group Members and Responsibilities

Member	Responsibility	Checkpoint 1 contribution
Macharia Grace Wairimu	Project concept, purpose and introduction	Project description, README update, and this report
Mwangi Billiart B. Muthoni	Mathematics, route geometry and graphics algorithms	Bresenham line, midpoint circle, Bézier curves and splines; repository setup and integration
Njugi Elijah Muiru	Core C++ programming, simulation and integration	Polygon representation and transformations; combined junction scene
Kihumba Kevin Renana	Scene design, interface and feature scope	Scan-line polygon fill; midpoint ellipse (trees, roundabout)
Kisotu Samuel Lemayian	Documentation, testing and demonstration	Sutherland–Hodgman polygon clipping; reviewing and approving the pull request

3. Technologies and Tools

- C++17: all algorithms and the image writer
- g++ (MSYS2 MinGW-w64): compiler
- PowerShell build script (scripts/build.ps1): compiles, tests, and renders in one command
- Git and GitHub: version control, issues, milestones, and pull requests
- Visual Studio Code: code editor

Images are written by a BMP file writer (src/canvas.cpp).

4. Project Structure

File	Purpose
include/graphics.hpp	Declarations of all graphics algorithms and shape types
src/graphics.cpp	Implementation of the algorithms (decides which pixels to colour)
include/canvas.hpp, src/canvas.cpp	Pixel grid that stores colours and saves them as a .bmp image
src/render.cpp	Builds the junction scenes and saves the checkpoint images
tests/graphics_test.cpp	Automated tests for every algorithm
scripts/build.ps1	Compiles everything, runs the tests, then renders the images

5. How to Compile and Run

Requirements: Windows with an x64 MinGW-w64 g++ compiler (for example, MSYS2 at C:\msys64\ucrt64\bin).

1. Clone the repository:
git clone https://github.com/BilliartMuthoni/smartflow-cpp-interactive-2D-traffic-simulation.git
2. In the project folder, run the build script:
powershell -NoProfile -ExecutionPolicy Bypass -File scripts/build.ps1
3. The script compiles the code, runs all tests, and saves nine images to the output folder.
Open the folder with: start output

6. Build and Automated Tests

The C++ algorithms are compiled with g++ (C++17) and checked by automated tests before the images are rendered. There is one group of tests per algorithm; if any check fails, the build stops.

```
PS C:\smart-flow\smartflow-cpp-interactive-2D-traffic-simulation> powershell -NoProfile -ExecutionPolicy Bypass -File scripts/build.ps1
Compiler: C:/msys64/ucrt64/bin/g++.exe
Building graphics tests
Bresenham: all tests passed
Midpoint circle: all tests passed
Cohen-Sutherland clipping: all tests passed
Polygon transformations: all tests passed
Scan-line fill: all tests passed
Sutherland-Hodgman clipping: all tests passed
Midpoint ellipse: all tests passed
Bezier curves and Catmull-Rom splines: all tests passed
All graphics tests passed
Building renderer
```

Figure 1: Build output: all eight algorithm test groups pass

7. Graphics Techniques Implemented

7.1 Line Drawing: Bresenham's Algorithm

What it does: Draws a straight line between two points using only whole-number arithmetic. An error value decides whether the next pixel steps sideways, so no decimals are needed. It works in all eight directions.

Use in SmartFlow: Road edges, dashed centre lines, and stop lines of the junction. The blue fan of lines shows the algorithm working in every direction.

Testing: Checks exact pixels for a known line, single-point lines, steep lines, and that lines have no gaps.

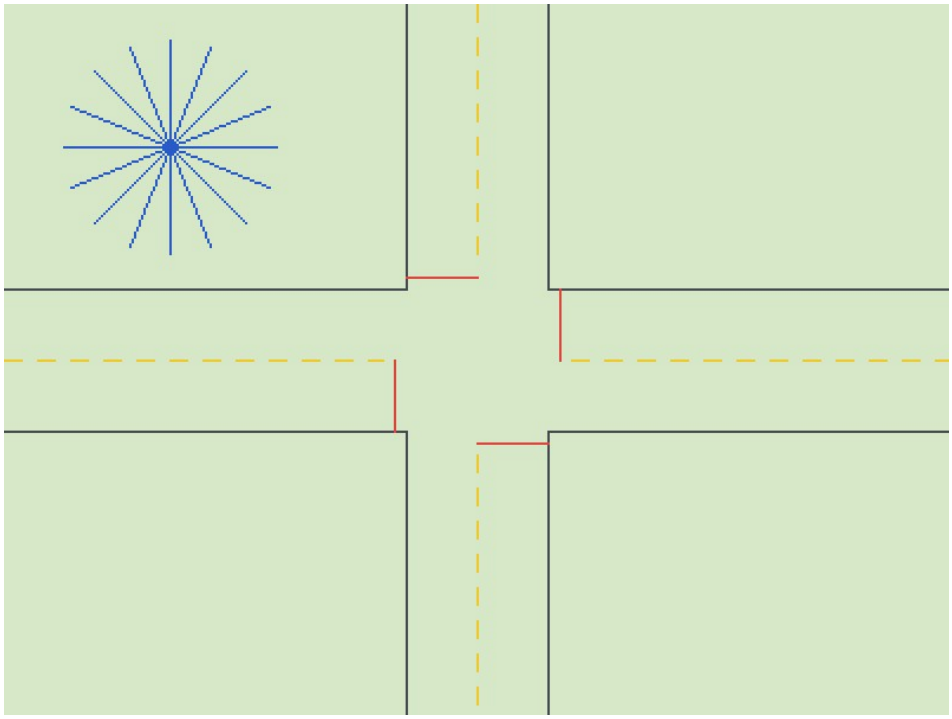


Figure 2: Junction road markings drawn with Bresenham lines

7.2 Line Clipping: Cohen–Sutherland

What it does: Keeps only the part of a line inside a rectangular window. Each endpoint gets a 4-bit region code showing which side of the window it is on. Lines fully inside are accepted, lines fully on one side are rejected, and the rest are cut at the window edge.

Use in SmartFlow: Vehicle paths are clipped to the junction viewport. Green shows the visible part kept, grey the part removed, and red lines rejected completely.

Testing: 7 tests: lines inside, crossing, outside, touching the edge, missing a corner, reversed direction, and invalid input.

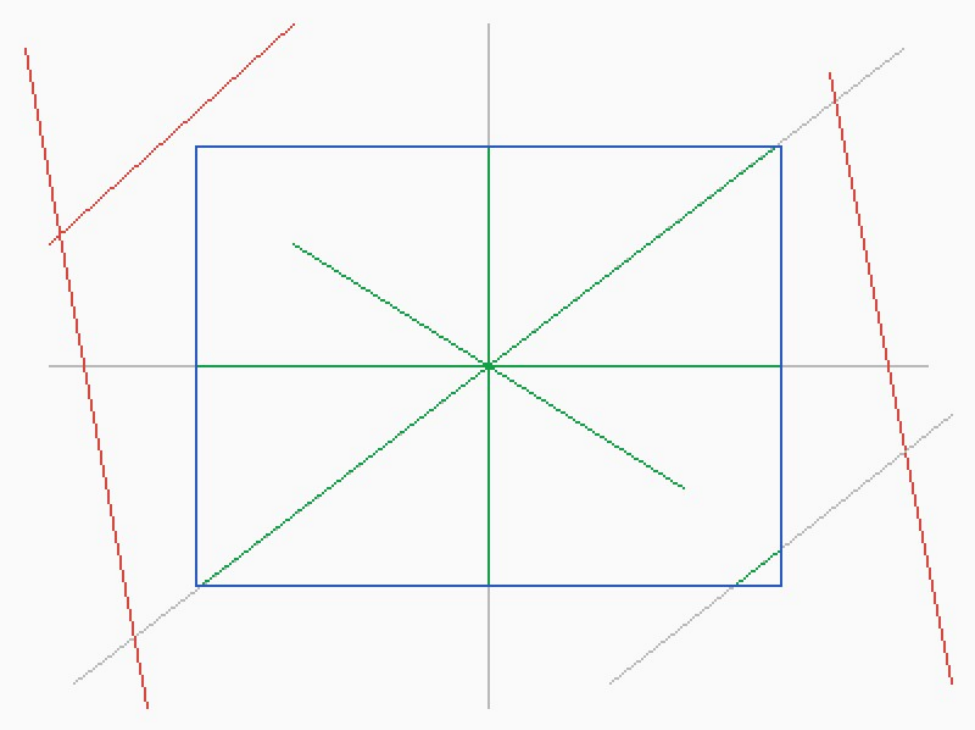


Figure 3: Vehicle paths clipped to the junction viewport

7.3 Circle: Midpoint Circle Algorithm

What it does: Calculates the pixels of one eighth of a circle using a decision value, then mirrors them into the other seven parts (eight-way symmetry).

Use in SmartFlow: Traffic-light lamps for the four junction approaches. Lit lamps are filled; unlit lamps are outlines.

Testing: Checks the number of pixels, accuracy to the true radius, no duplicate pixels, symmetry, and special cases (radius 0 and negative radius).

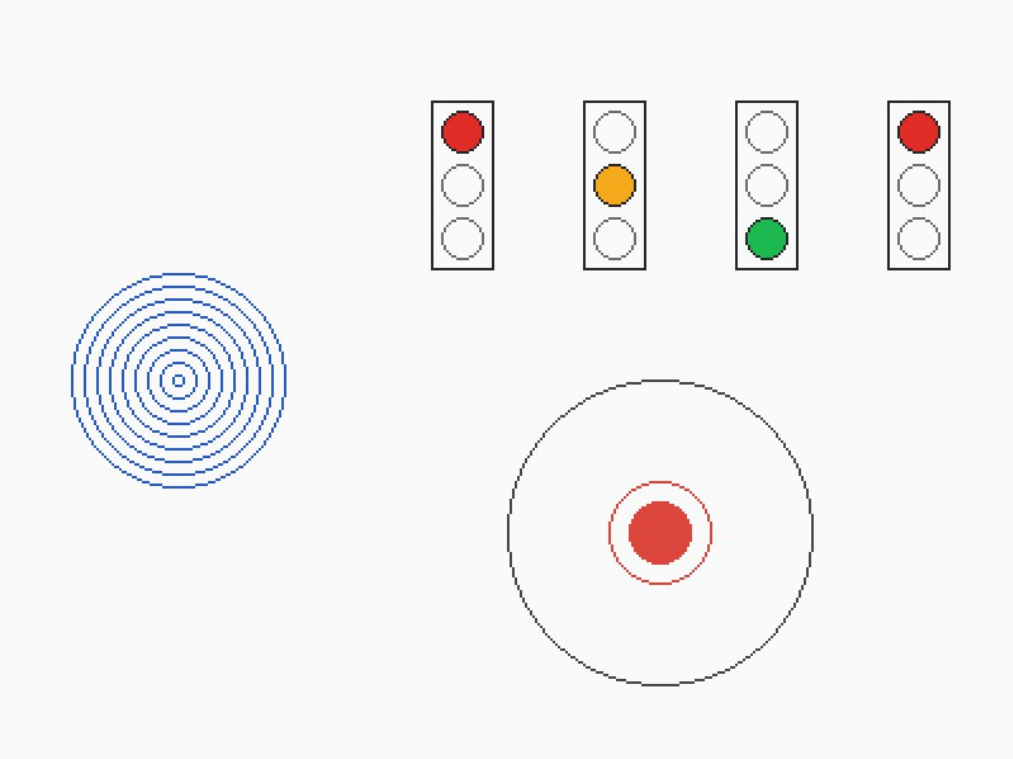


Figure 4: Traffic-light lamps drawn with the midpoint circle algorithm

7.4 Polygon Representation and Transformations

What it does: A polygon is stored as an ordered list of corner points. We implemented translation (moving), rotation about a pivot, and scaling, plus the shoelace formula for area.

Use in SmartFlow: A car body is rotated and moved through a left turn at the junction. Buildings are scaled and rotated.

Testing: 6 tests: area, translation, 90° rotation, rotation about the centre, scaling, and a closed outline.

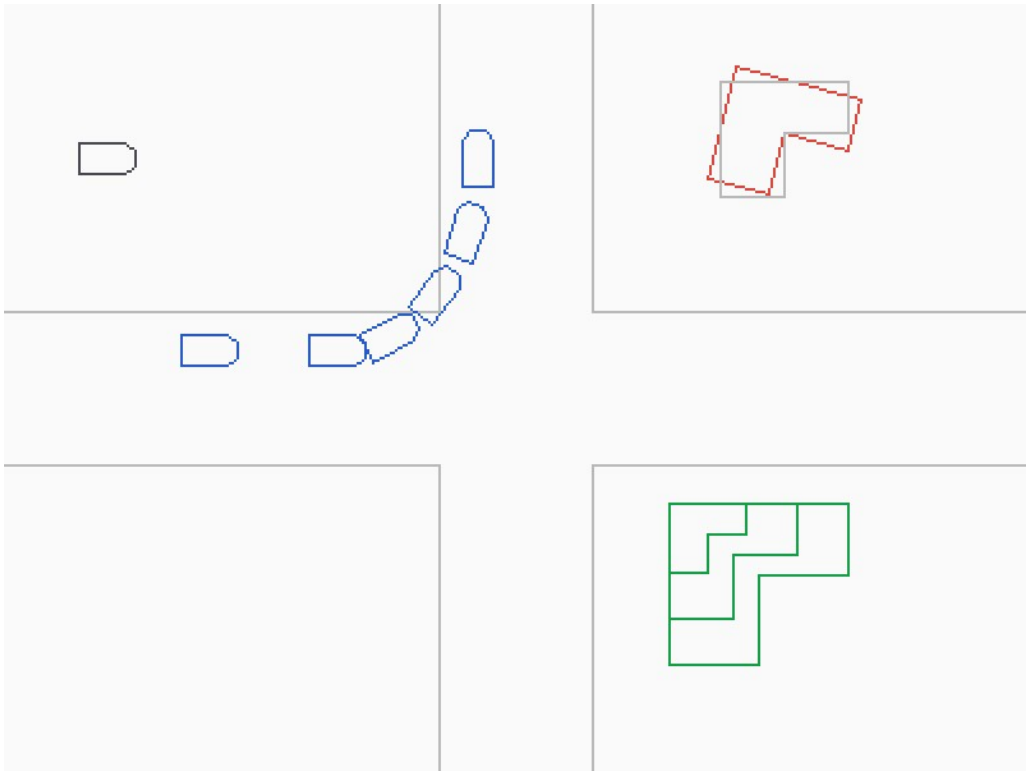


Figure 5: A car rotated through a left turn, and buildings scaled and rotated

7.5 Polygon Filling: Scan-Line Algorithm

What it does: For each row of pixels, the algorithm finds where the row crosses the polygon's edges, sorts those points from left to right, and fills between each pair. This also works for concave (non-convex) shapes.

Use in SmartFlow: Roads, L-shaped buildings, a star-shaped plaza, and the cars are filled.

Testing: 6 tests: rectangle pixel count, vertex order, a concave L-shape with an empty notch, no repeated pixels, a triangle, and invalid input.

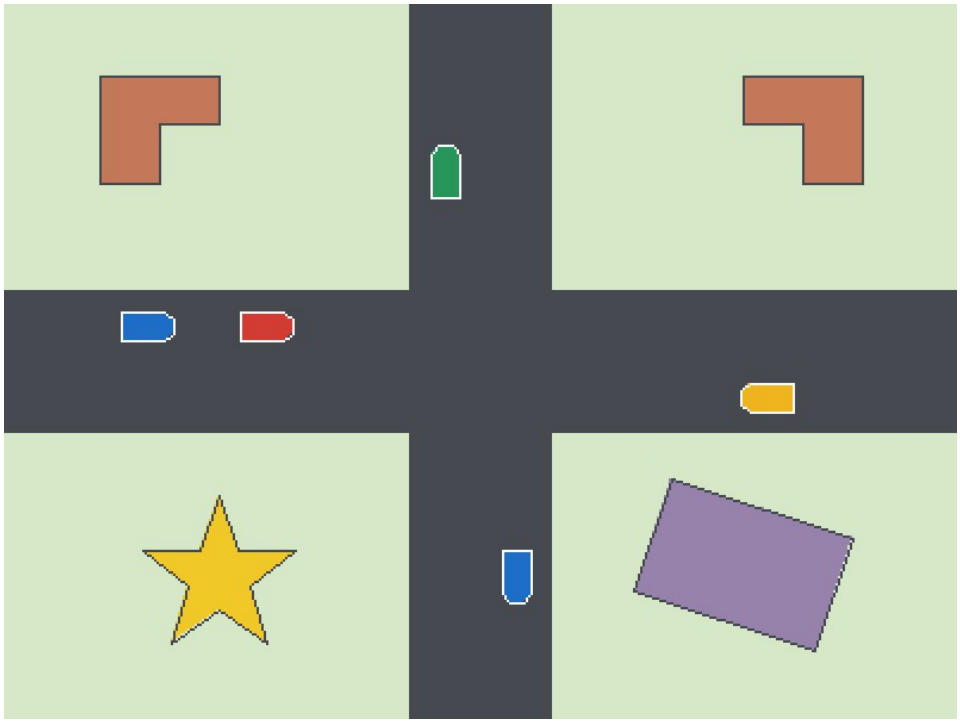


Figure 6: Roads, buildings and cars filled with the scan-line algorithm

7.6 Polygon Clipping: Sutherland–Hodgman

What it does: Clips a polygon against each edge of a rectangular window in turn. For every edge of the polygon, it keeps points inside the window and adds new points where the polygon crosses the window edge.

Use in SmartFlow: Cars and buildings that cross the junction viewport are cut at its edge; shapes completely outside are removed.

Testing: 6 tests: fully inside, fully outside, half outside, larger than the window, a corner cut, and a concave shape.

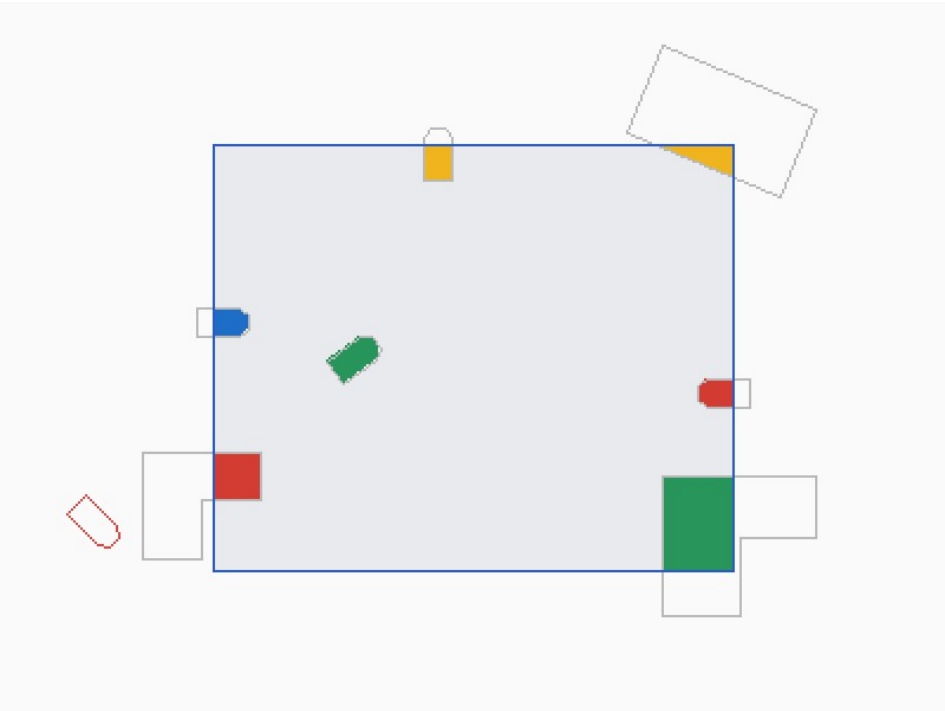


Figure 7: Cars and buildings clipped at the junction viewport

7.7 Ellipse: Midpoint Ellipse Algorithm

What it does: Draws an ellipse in two regions: where the curve is flatter, it steps along x, and where it is steeper, it steps along y. Each calculated point is mirrored into all four quarters.

Use in SmartFlow: The roundabout island and the tree canopies. The blue set of ellipses shows wide, round, and tall shapes.

Testing: Checks the four extreme points, symmetry, accuracy, that the outline has no gaps, and flat (zero-radius) cases.

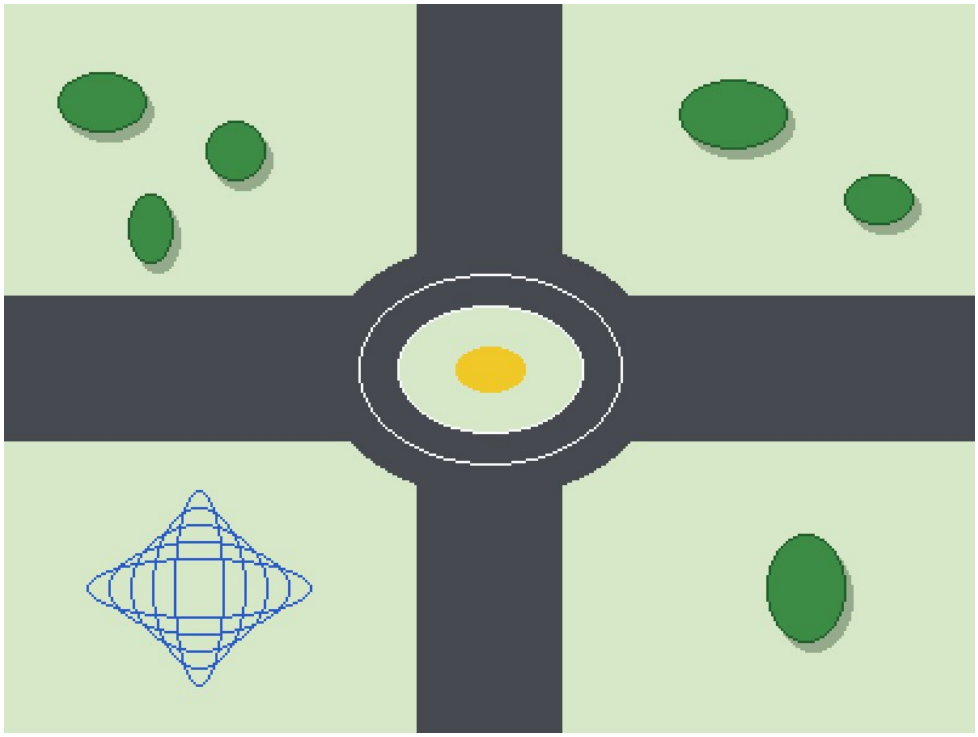


Figure 8: Roundabout and tree canopies drawn with the midpoint ellipse algorithm

7.8 Curves: Bézier Curves and Catmull–Rom Splines

What it does: A cubic Bézier curve uses four control points: it starts at the first, ends at the last, and bends towards the two middle points. A Catmull–Rom spline is a smooth curve that passes through every control point.

Use in SmartFlow: Bézier curves give the turning routes of vehicles through the junction (yellow points show the control points of one route). Catmull–Rom splines draw a winding side road and a footpath through their red control points.

Testing: 7 tests: start and end points, a value calculated by hand, straight-line case, sample count, and the spline passing through every control point.

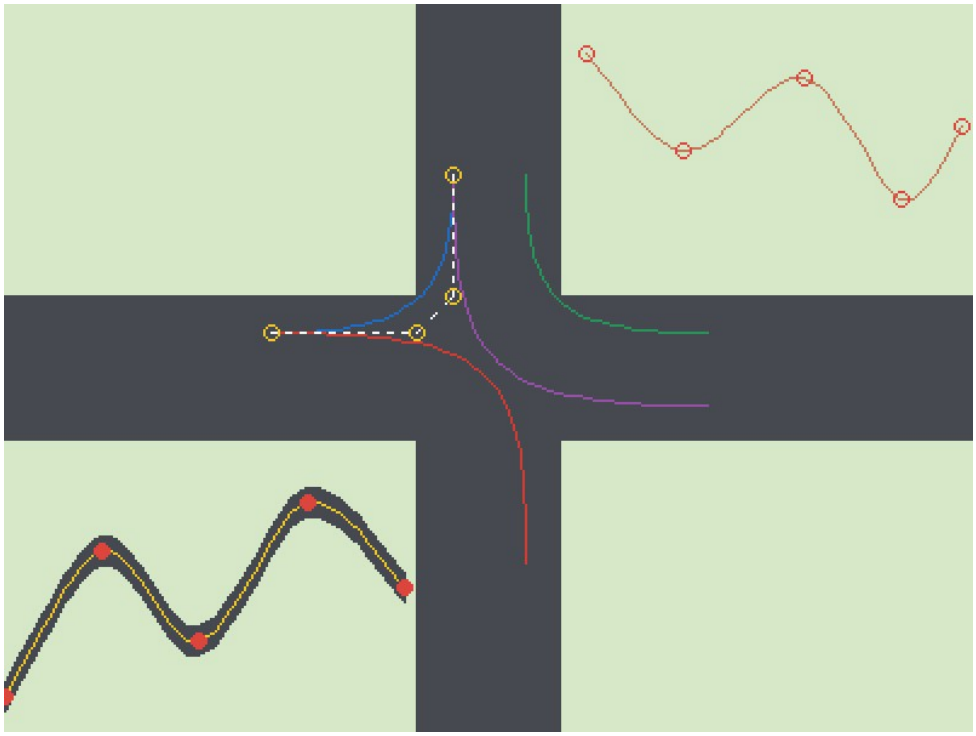


Figure 9: Bézier turning routes and Catmull–Rom spline roads

7.9 Combined Junction Scene

What it does: Brings all the techniques together into one picture of the SmartFlow junction.

Use in SmartFlow: Filled roads and buildings, Bresenham road markings, midpoint-circle signal lamps, ellipse trees, cars placed and rotated along Bézier routes, a spline side road, and both clipping algorithms applied at the viewport edge.

Testing: Built from the tested algorithms above.

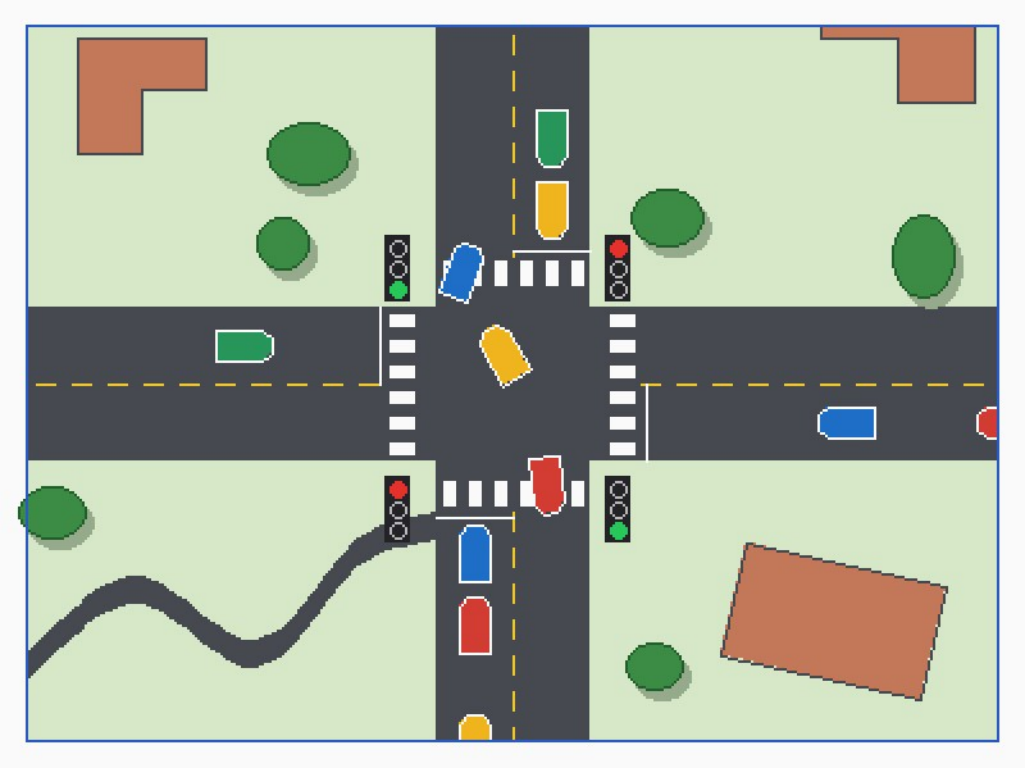


Figure 10: The SmartFlow junction built from all implemented techniques

8. Development History

Work was planned as GitHub issues under the Checkpoint 1 milestone. Each commit closes one issue, and the work was merged into the main branch through a reviewed pull request.

9. Current Progress

- All eight required topics are implemented in pure C++: line drawing, line clipping, circles, polygon representation, polygon filling, polygon clipping, ellipses, and Bézier curves and splines.
- Every algorithm has automated tests, and all tests pass.
- Each algorithm is applied to a part of the SmartFlow junction, and a combined scene shows them working together.
- The repository has a clear structure, issues for each task, and a commit history recording the development.

Next milestones: animate vehicles moving along the routes, add traffic-signal timing, and build the interactive window and controls.

10. Known Limitations

- The output is still images, not an animation or interactive window; animation and interaction are planned for later milestones.
- The build script is written for Windows and needs a MinGW-w64 g++ compiler.
- Images are saved as BMP files, which are larger than PNG files.
- Lines are one pixel wide, and edges are not smoothed (no anti-aliasing).
- Polygon clipping works against rectangular windows only.
- The scene is a simplified model of a single junction and is not based on real traffic data.